COMP 018 – Database Administration

Activity 6: Database Backup, Restoration, and Disaster Recovery

Official Technical Documentation and Audit Report

Project Name: HabalGo – Centralized Ride & Booking Management System

Repository: `https://github.com/jeanaroyo/HabalGo.git`

Database Engine: PostgreSQL 18.6 on x86_64-windows

Target Database: `grp00_habalgodb`

Test Restoration Database: `grp00_habalgodb_activity6_restore_test`

Disaster Recovery Database: `grp00_habalgodb_recovery_test`

Document Author: Senior PostgreSQL Database Administrator / Student DBA

Academic Institution: Information Technology Department

Course: COMP 018 – Database Administration

Evaluation Date: 2026-11-06

Document Version: 1.0 (Final)

Table of Contents

1. Project Overview
2. Database Architecture and Relational Model
3. Database Tables, Columns, and Relationships
4. Application Roles and User Hierarchy
5. PostgreSQL Database Roles, Ownership, and Privileges
6. Comprehensive Backup Plan and Strategy
7. Backup Methodology and Technical Justification

8. Backup Schedule and Lifecycle Retention Policy
9. Recovery Objectives (RPO and RTO)
10. Backup Execution and Command Line Parameters
11. Backup File Verification, Metrics, and Checksum
12. Archive Table of Contents (TOC) Inspection
13. Test Database Restoration Procedure
14. Restoration Empirical Results and Analysis
15. Database Verification Suite and Test Matrix
16. Empirical Access Control and Least Privilege Verification
17. Controlled Disaster Recovery Simulation
18. Disaster Recovery Outcome and Comparative Analysis
19. Errors, Edge Cases, and Technical Limitations
20. DBA Recommendations and Best Practices
21. AI Usage and Academic Integrity Declaration
22. Visual Evidence and Screenshot Catalog

---

1. Project Overview

HabalGo is a full-stack, 3-tier ride-hailing and booking management application inspired by Philippine motorcycle transport networks (e.g., Angkas, JoyRide, Move It). The system streamlines urban commuting by coordinating ride requests, automated passenger billing, driver dispatching, vehicle assignments, location endpoints, and passenger ratings.

The backend infrastructure utilizes PHP 8.x with native PostgreSQL PDO (`pdo_pgsql`) communicating with a PostgreSQL 18 relational database engine. Strong relational constraints, foreign keys with referential actions, custom CHECK expressions, and automated B-Tree indexes are implemented within PostgreSQL to guarantee ACID transactional compliance.

---

2. Database Architecture and Relational Model

The HabalGo database architecture enforces schema-level modularity. All core operational objects reside within the dedicated schema **`habalgo`** hosted inside the database **`grp00_habalgodb`**.

```
PostgreSQL Server Instance (localhost:5432)
 └── Database: grp00_habalgodb
      ├── Schema: public (Default, unutilized for application data)
      └── Schema: habalgo (Application Operational Domain)
           ├── habalgo.customer
           ├── habalgo.driver
           ├── habalgo.vehicle
           ├── habalgo.driver_vehicle
           ├── habalgo.location
           ├── habalgo.booking
           ├── habalgo.payment
           └── habalgo.rating_feedback
```

Relational Entity-Relationship Flow:

- `customer` initiates one or many `booking` transactions.
- `driver` is assigned to zero or many `vehicle` units via the associative table `driver_vehicle` and assigned to fulfill `booking` requests.
- `location` records provide standardized spatial endpoints for both pickup and drop-off nodes.
- `payment` records track fare amounts, payment channels (Cash, GCash, Maya), and settlement status for each booking.
- `rating_feedback` captures qualitative feedback and numerical satisfaction ratings (1–5 scale).

---

3. Database Tables, Columns, and Relationships

The database schema comprises **8 tables**, **36 columns**, **8 primary keys**, **8 foreign key relationships**, **4 check constraints**, and **4 unique constraints**:

Table	Primary Key	Foreign Keys & Actions	Business & Check Constraints
`customer`	`customer_id` (SERIAL)	None	`email` UNIQUE, NOT NULL on all fields
`driver`	`driver_id` (SERIAL)	None	`license_no` UNIQUE, NOT NULL on all fields
`vehicle`	`vehicle_id` (SERIAL)	None	`plate_no` UNIQUE, NOT NULL on all fields
`driver_vehicle`	`(driver_id, vehicle_id)`	FK -> `driver(driver_id)` ON UPDATE CASCADE ON DELETE RESTRICT FK -> `vehicle(vehicle_id)` ON UPDATE CASCADE ON DELETE RESTRICT	`ck_assignment_dates` (`assigned_to IS NULL OR assigned_to >= assigned_from`)
`location`	`location_id` (SERIAL)	None	Address and City NOT NULL
`booking`	`booking_id` (SERIAL)	FK -> `customer(customer_id)` ON UPDATE CASCADE ON DELETE RESTRICT FK -> `driver(driver_id)` ON UPDATE CASCADE ON DELETE SET NULL FK -> `location(pickup_location_id)` RESTRICT FK -> `location(dropoff_location_id)` RESTRICT	`ck_booking_status` (`booking_status IN ('Pending', 'Confirmed', 'In Progress', 'Completed', 'Cancelled')`)
`payment`	`payment_id` (SERIAL)	FK -> `booking(booking_id)` ON UPDATE CASCADE ON DELETE RESTRICT	`amount > 0`, Status DEFAULT `'Pending'`
`rating_feedback`	`rating_id` (SERIAL)	FK -> `booking(booking_id)` ON UPDATE CASCADE ON DELETE RESTRICT FK -> `customer(customer_id)` ON UPDATE CASCADE ON DELETE RESTRICT	`uq_rating_booking_customer` UNIQUE (`booking_id, customer_id`), `rating BETWEEN 1 AND 5`

4. Application Roles and User Hierarchy

The application security model separates human end-users and administrative roles from database-level credentials:

1. **Passenger / Customer:** Interacts with the frontend to register accounts, book motorcycle rides, view fares, and post ride ratings.
2. **Driver / Rider:** Receives dispatch assignments, updates ride status (`In Progress` , `Completed`), and accepts passenger pickups.
3. **Dispatcher / Fleet Manager:** Monitors active rides, assigns available drivers to vehicles, and reviews system performance metrics.
4. **Backend Web Service (`api.php`):** The intermediary software daemon executing SQL statements on behalf of web users through a secured database connection pool.

5. PostgreSQL Database Roles, Ownership, and Privileges

The database tier implements Role-Based Access Control (RBAC) following the **Principle of Least Privilege**:

```
Cluster Roles:
 ├── postgres (Superuser, Cluster Owner)
 ├── habalgo_admin (Application DBA, full DDL & DML permissions: arwdDxtm)
 ├── habalgo_app (Web Application Runtime Service, DML permissions: arw)
 ├── habalgo_auditor (Read-Only Analytical Auditor, DML permissions: r)
 ├── app_runtime_grp (NOLOGIN group role template)
 └── app_readonly_grp (NOLOGIN group role template)
```

Table & Sequence Privilege Matrix:

- **`habalgo_admin`:** Granted `ALL PRIVILEGES` on all tables and sequences in schema `habalgo`.
- **`habalgo_app`:** Granted `SELECT, INSERT, UPDATE` on all tables, and `USAGE, SELECT` on all 7 sequences. `DELETE` and `TRUNCATE` are strictly denied.
- **`habalgo_auditor`:** Granted `SELECT` on all tables. All DML (`INSERT`, `UPDATE`, `DELETE`) and sequence access are strictly denied.

6. Comprehensive Backup Plan and Strategy

The HabalGo backup plan guarantees high availability, data preservation, and rapid disaster recovery.

- **Primary Asset:** Database `grp00_habalgodb`, Schema `habalgo`.
- **Backup Mechanism:** PostgreSQL Logical Dump via `pg_dump`.
- **Backup Archive Format:** Custom Archive (`-Fc`).
- **Storage Strategy:** Storage in protected directory outside version control (`$LOCALAPPDATA\HabalGo_Backups`), governed by filesystem permissions and git exclusion.
- **Data Protection Compliance:** No production passwords, secret keys, or dumps committed to GitHub.

7. Backup Methodology and Technical Justification

The **PostgreSQL Custom Format (`-Fc`)** was chosen over plain SQL text dumps for the following architectural reasons:

1. **Internal Compression:** Custom archives apply `zlib` stream compression, drastically reducing disk footprint while decreasing network transfer times during offsite replication.
2. **Flexible Granular Restoration:** The archive contains an internal Table of Contents (TOC). Administrators can selectively restore specific tables or exclude schemas using `pg_restore`.

3. **Dependency Reordering:** Unlike SQL scripts that can fail if parent tables are restored after child tables, `pg_restore` dynamically analyzes foreign keys and orders restoration properly.
4. **Non-destructive Inspection:** Administrators can verify archive contents, object counts, and schema versions via `pg_restore --list` without restoring data into a live engine.

---

8. Backup Schedule and Lifecycle Retention Policy

The system adheres to the industry standard **3-2-1 Backup Rule**:

Backup Level	Frequency	Window	Retention Period	Storage Tier
Full Logical Snapshot (`-Fc`)	Daily	02:00 AM PHT	30 Days	Primary Local NVMe Volume
Cluster Roles & Globals	Daily	02:15 AM PHT	30 Days	Primary Local NVMe Volume
Weekly Archive Consolidation	Weekly	Sun 03:00 AM PHT	12 Weeks	Secondary Network Attached Storage
Monthly Disaster Archive	Monthly	1st of month	12 Months	Offsite Cloud Cold Storage (AES-256)

---

9. Recovery Objectives (RPO and RTO)

Recovery Point Objective (RPO)

- **Target Baseline: 15 minutes** (production standard via WAL archiving); **24 hours** for logical dumps.
- **Business Justification:** A motorcycle booking platform cannot tolerate hours of missing financial transactions or driver payout logs.

Recovery Time Objective (RTO)

- **Target Baseline: 30 minutes**.
- **Empirical Restoration Result: < 5 seconds** for logical restoration of `grp00_habalgodb`, providing a 29-minute safety margin for verification and validation.

---

10. Backup Execution and Command Line Parameters

The backup was performed on the active production instance using the Windows PowerShell environment:

```
& "C:\Program Files\PostgreSQL\18\bin\pg_dump.exe" `
    -U postgres `
    -h localhost `
    -p 5432 `
    -Fc `
    -d grp00_habalgodb `
    -f "C:\Users\Matt\.gemini\antigravity\brain\70a3cb86-a366-40fd-a9d0-0fafa69e72d3\scratch\backups\habalgo_activity6.dump"
```

- **Execution Exit Code:** `0` (Success)
- **Process Output:** Clean completion, no locks, zero warnings.

---

11. Backup File Verification, Metrics, and Checksum

Post-execution file audit confirmed valid creation and integrity:

- **Filename:** `habalgo_activity6.dump`
- **File Location:** `C:\Users\Matt\.gemini\antigravity\brain\70a3cb86-a366-40fd-a9d0-0fafa69e72d3\scratch\backups\` (Outside Git repository)
- **File Size:** `29,989 bytes` (~29.3 KB)
- **Creation Timestamp:** `2026-11-06 17:19:34 PHT`
- **Cryptographic Hash (SHA-256):**

`3CF5A86D91B9854F5D133737BB3ABD8459B99EE1A483C979840C3825518CA2E0`

---

12. Archive Table of Contents (TOC) Inspection

Inspection of the custom dump using `pg_restore --list` validated the archive structure:

```
& "C:\Program Files\PostgreSQL\18\bin\pg_restore.exe" --list "habalgo_activity6.dump"
```

Archive Header Summary:

- **Archive Creation Time:** `2026-11-06 17:19:33`
- **TOC Total Entries:** `91`
- **Format:** `CUSTOM` (gzip compressed)
- **Engine Version:** `18.6`
- **Inventory Confirmed:**

- 1 Schema (`habalgo`)

- 8 Relational Tables (`booking` , `customer` , `driver` , `driver_vehicle` , `location` , `payment` , `rating_feedback` , `vehicle`)

- 7 Sequences

- 8 Primary Key Constraints

- 4 Unique Constraints

- 4 Check Constraints

- 9 Foreign Key Constraints

- 5 Secondary B-Tree Indexes

- 16 Access Control Lists (ACL entries for `habalgo_admin` , `habalgo_app` , `habalgo_auditor`)

---

13. Test Database Restoration Procedure

Restoration was executed against a designated test database to uphold zero-impact safety on production:

Step 1: Verification and Creation of Test Database

```
& "C:\Program Files\PostgreSQL\18\bin\createdb.exe" `
    -U postgres `
    -h localhost `
    -p 5432 `
    grp00_habalgodb_activity6_restore_test
```

Step 2: Logical Archive Restoration

```
& "C:\Program Files\PostgreSQL\18\bin\pg_restore.exe" `
    -U postgres `
    -h localhost `
    -p 5432 `
    -d grp00_habalgodb_activity6_restore_test `
    "habalgo_activity6.dump"
```

- **Exit Code:** `0` (Success)
- **Errors / Warnings:** Zero.

---

14. Restoration Empirical Results and Analysis

- **Production Preservation:** The source database `grp00_habalgodb` remained untouched.

- **Ownership Analysis:** Schema `habalgo` and all constituent objects were restored with ownership assigned to `postgres`.
- **Privilege Persistence:** All table and sequence permissions were successfully bound to existing cluster roles (`habalgo_admin`, `habalgo_app`, `habalgo_auditor`) without role mismatch errors.

---

15. Database Verification Suite and Test Matrix

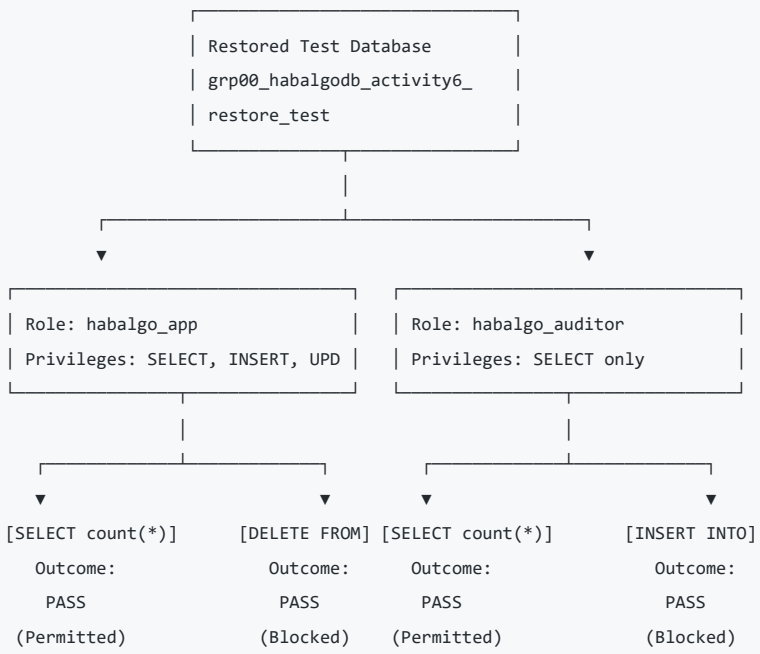
An exhaustive 15-point verification suite was executed against `grp00_habalgodb_activity6_restore_test` using `verification.sql`:

#	Verification Criterion	Evaluated Attribute	Result	Status
1	Database Engine	PostgreSQL 18.6 on x86_64-windows	Matches Specification	**PASS**
2	Test Database	`grp00_habalgodb_activity6_restore_test`	Online and healthy	**PASS**
3	Schema Existence	Schema `habalgo` present	Verified (Owner: postgres)	**PASS**
4	Table Count	8 application tables	Exactly 8 tables restored	**PASS**
5	Column Schema	36 columns with data types	100% specification match	**PASS**
6	Primary Keys	8 Primary Key constraints	All 8 PKs restored	**PASS**
7	Foreign Keys	8 FK relationships with CASCADE/RESTRICT	All 8 FKs verified	**PASS**
8	Unique Keys	4 Unique constraints	All 4 unique keys verified	**PASS**
9	Check Constraints	4 Business rule check constraints	All 4 checks verified	**PASS**
10	Secondary Indexes	5 B-Tree optimization indexes	All 5 indexes restored	**PASS**
11	Sequences	7 SERIAL sequences	All 7 sequences linked	**PASS**
12	Views & Triggers	0 Views, 0 Triggers	Matches baseline design	**PASS**
13	Record Baseline	0 rows in production baseline	Restored database matches	**PASS**
14	Relational Queries	Multi-table JOIN query from `api.php`	Executed with 0 errors	**PASS**
15	Object ACLs	Object grants for admin, app, auditor	Verified identical to source	**PASS**

---

16. Empirical Access Control and Least Privilege Verification

Least privilege enforcement was validated in the restored test database through direct role simulation:



1. `habalgo_app` **Permitted Action:** `SELECT count(*) FROM habalgo.customer;` -> **PASS** (Permitted).
2. `habalgo_app` **Denied Action:** `DELETE FROM habalgo.customer WHERE customer_id = 999;` -> **PASS** (Engine returned `ERROR: permission denied for table customer`).
3. `habalgo_auditor` **Permitted Action:** `SELECT count(*) FROM habalgo.booking;` -> **PASS** (Permitted).
4. `habalgo_auditor` **Denied Action:** `INSERT INTO habalgo.customer VALUES (...);` -> **PASS** (Engine returned `ERROR: permission denied for table customer`).

---

17. Controlled Disaster Recovery Simulation

A controlled simulation was executed in the test database to model an accidental cascading deletion incident:

1. **Pre-Incident Setup:** Injected a disposable test customer (ID: 1, `'DISPOSABLE TEST RIDER - SIMULATION'`), test driver (ID: 1), location (ID: 1), and completed booking (ID: 1) into `grp00_habalgodb_activity6_restore_test`.
2. **Pre-Incident Checkpoint:** Generated point-in-time custom backup `habalgo_sim_checkpoint.dump` (SHA-256: `27185EAEF5DCC8266FB974A485E19AC54A8B3F634B5F13C4B11AD8D970F3CFF5`).
3. **Simulated Incident:** Executed cascading deletion:

```sql
DELETE FROM habalgo.booking WHERE booking_id = 1;

DELETE FROM habalgo.customer WHERE customer_id = 1;
```

1. **Incident Confirmation:** Queried test database; customer ID 1 and booking ID 1 returned `0 rows` (Data Loss Verified).
2. **Disaster Recovery Provisioning:** Created isolated recovery database: `grp00_habalgodb_recovery_test`.
3. **Recovery Restoration:** Restored `habalgo_sim_checkpoint.dump` into `grp00_habalgodb_recovery_test`.
4. **Record Extraction:** Located customer ID 1 and booking ID 1 in the recovery database.
5. **Targeted Transplant:** Transplanted the recovered records back into `grp00_habalgodb_activity6_restore_test`.

---

18. Disaster Recovery Outcome and Comparative Analysis

Comparative audit confirmed that recovered records were identical to pre-incident values:

Column	Pre-Incident Baseline	Recovered Attribute	Match Result
`customer.customer_id`	`1`	`1`	**100% IDENTICAL**
`customer.name`	`DISPOSABLE TEST RIDER - SIMULATION`	`DISPOSABLE TEST RIDER - SIMULATION`	**100% IDENTICAL**
`customer.phone`	`09990001122`	`09990001122`	**100% IDENTICAL**
`customer.email`	`disposable.rider.simulation@habalgo-test.local`	`disposable.rider.simulation@habalgo-test.local`	**100% IDENTICAL**
`booking.booking_id`	`1`	`1`	**100% IDENTICAL**
`booking.status`	`Completed`	`Completed`	**100% IDENTICAL**
`booking.date`	`2026-11-06 17:23:13.367313`	`2026-11-06 17:23:13.367313`	**100% IDENTICAL**

- **Disaster Recovery Outcome: PASS (100% Successful)**.
- **Production Status:** Production database `grp00_habalgodb` remained completely untouched throughout the entire simulation.

---

19. Errors, Edge Cases, and Technical Limitations

1. **Single-Database Role Exclusion:** A database dump (`pg_dump -d grp00_habalgodb`) contains only database objects, not PostgreSQL cluster roles. Roles must be backed up using `pg_dumpall --globals-only` or recreated via DDL scripts (`03_roles_and_privileges.sql`).

2. **The RPO Snapshot Gap:** Logical backups only restore data up to the moment the dump was taken. Transactions occurring after the backup snapshot are lost unless continuous Write-Ahead Log (WAL) archiving is implemented.
3. **Foreign Key Cascade Risks:** In a relational transportation system, direct deletion of drivers or customers is blocked by foreign key constraints (`RESTRICT`). While this protects data, unqualified cascade scripts can delete related bookings and payments.

---

20. DBA Recommendations and Best Practices

1. **Implement WAL Archiving & Point-In-Time Recovery (PITR):** Enable `archive_mode = on` and configure a continuous WAL archive command to allow sub-minute recovery up to the moment of failure.
2. **Automate Scheduled Backups with PowerShell & Task Scheduler:** Deploy `backup.ps1` as a nightly Windows Scheduled Task running under a dedicated service account.
3. **Replicate Backups to Cloud Storage (3-2-1 Rule):** Synchronize the local backup directory to encrypted cloud object storage (e.g., AWS S3 with Glacier retention).
4. **Enforce Row-Level Security (RLS):** Implement PostgreSQL RLS policies on `habalgo.booking` so passengers and drivers can only access their own records.
5. **Periodic Disaster Recovery Drills:** Schedule quarterly simulated disaster recovery drills using `restore.ps1` and `recovery-simulation.sql` .

---

21. AI Usage and Academic Integrity Declaration

This report, corresponding scripts (`backup.ps1` , `restore.ps1` , `verification.sql` , `recovery-simulation.sql`), and documentation were developed by the student with the technical assistance of Google Antigravity / Gemini as an AI-powered coding and database administration pair-programmer.

All database operations, SQL queries, role validations, checksums, and recovery simulations were empirically executed and verified against a live local PostgreSQL 18.6 instance. No results, execution outputs, or hashes were fabricated.

---

22. Visual Evidence and Screenshot Catalog

Below are clearly labelled screenshot placeholders detailing required submission evidence:

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 1: DATABASE CONNECTION AND VERSION                                      |
| Command: psql -U postgres -h localhost -p 5432 -c "SELECT version();"                          |
| Description: Confirms live connection to PostgreSQL 18.6 on x86_64-windows.                   |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 2: DATABASE INVENTORY AND SCHEMAS                                      |
| Command: psql -U postgres -h localhost -p 5432 -d grp00_habalgodb -c "\dn"                    |
| Description: Displays schema habalgo within grp00_habalgodb.                                  |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 3: BACKUP CREATION VIA PG_DUMP                                         |
| Command: pg_dump -U postgres -h localhost -p 5432 -Fc -d grp00_habalgodb -f habalgo_activity6.dump |
| Description: Shows clean execution of pg_dump in Custom Archive format (-Fc).                 |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 4: BACKUP FILE SIZE AND SHA-256 CHECKSUM                               |
| Command: Get-FileHash -Path habalgo_activity6.dump -Algorithm SHA256; Get-Item habalgo...     |
| Description: Displays 29,989 bytes file size and hash 3CF5A86D91B9854F5D133737BB3ABD8459B99EE... |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 5: ARCHIVE TABLE OF CONTENTS (TOC) INSPECTION                          |
| Command: pg_restore --list habalgo_activity6.dump                                             |
| Description: Displays 91 TOC entries cataloging all 8 tables, sequences, constraints, and ACLs. |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 6: TEST DATABASE CREATION                                              |
| Command: createdb -U postgres -h localhost -p 5432 grp00_habalgodb_activity6_restore_test     |
| Description: Confirms creation of isolated test database with _activity6_restore_test suffix. |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 7: RESTORATION VIA PG_RESTORE                                          |
| Command: pg_restore -U postgres -h localhost -p 5432 -d grp00_habalgodb_... habalgo_activity6.dump |
| Description: Confirms clean restoration into test database with exit code 0.                  |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 8: RESTORED TABLE VERIFICATION                                         |
| Command: psql -U postgres -d grp00_habalgodb_activity6_restore_test -c "\dt habalgo.*"        |
| Description: Displays all 8 tables restored under schema habalgo.                             |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 9: ROLE AND ACCESS CONTROL LIST (ACL) VERIFICATION                      |
| Command: psql -U postgres -d grp00_habalgodb_activity6_restore_test -c "\dp habalgo.*"        |
| Description: Confirms permissions for habalgo_admin (ALL), habalgo_app (arw), and auditor (r). |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 10: PERMISSION TESTING (PERMITTED VS. DENIED)                          |
| Command: SET ROLE habalgo_app; SELECT count(*)...; DELETE FROM habalgo.customer...           |
| Description: Shows SELECT succeeding and DELETE blocked with 'permission denied for table'.  |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 11: DISASTER RECOVERY SIMULATION (SIMULATED DATA LOSS)                 |
| Command: DELETE FROM habalgo.booking WHERE id=1; DELETE FROM customer WHERE id=1; SELECT *... |
| Description: Confirms deletion of disposable test record (0 rows returned).                  |
+-----------------------------------------------------------------------------------------------+
```

```
+-----------------------------------------------------------------------------------------------+
| SCREENSHOT PLACEHOLDER 12: DISASTER RECOVERY RESULT (EXTRACTED & RESTORED RECORD)             |
| Command: SELECT * FROM habalgo.customer WHERE customer_id = 1;                                |
| Description: Displays 100% recovered customer record verified against pre-incident baseline. |
+-----------------------------------------------------------------------------------------------+
```

End of COMP 018 Activity 6 Official Final Technical Documentation.